```
raise any other errors")
    raise random.choice([IOError,Exception])
retry_io_error()
```

The output is shown in Figure 1.

In the above example, the retry is attempted only if *IOError* is raised. Likewise, there are multiple other conditional retry strategies like *retry_if_not_exception_type*, *retry_if_exception_message*, *retry_if_not_exception_type*, etc.

Changing retry behaviour based on function outcome

We can extend the retry condition based on the outcome of another function. This approach is of immense value, as we can try to implement a remediation strategy for the application for the retry attempt to be made.

```
import requests
import random
def validate_live():
    choice_result = [True,False]
    print("bring the choice of result")
    return random.choice(choice_result)
@retry(retry=retry_if_result(validate_live))
def execute_return_none():
    print("Retry with no wait if the return value is True")
execute_return_none()
    print("Retry with no wait if the return value is True")
execute_return_none()
```

Note that we haven't raised any exception in the above. However, the retry loop will be triggered based on the outcome of the function. In this example, it is *validate_live*.

The output will be:

```
/usr/local/bin/python3 /Users/Srikanth_Mohan/learning/learn_
tenacity.py
Retry with no wait if return value is True
bring the choice of result
Retry with no wait if return value is True
bring the choice of result
Retry with no wait if return value is True
bring the choice of result
Process finished with exit code 0
```

Retrying for code block

Many times, we would like the retry to be implemented within a code block rather than wrapping it at the function level.

```
from tenacity import Retrying, RetryError, stop_after_attempt
try:
    for attempt in Retrying(stop=stop_after_attempt(3)):
        with attempt:
            raise Exception('My code is failing!')
except RetryError:
    pass
```

Providing details on the retries

We can get the details on the retries made over a function by using the retry attribute attached to the function.

```
import tenacity
from tenacity import *
import random
@retry(retry=retry_if_exception_type(IOError),stop=stop_
after_attempt(3))
def retry_io_error():
    print("Retry forever with no wait if an IOError occurs,
raise any other errors")
    raise random.choice([IOError,Exception])
try:
 retry_io_error()
except Exception:
    pass

print (retry_io_error.retry.statistics)
```

The output will be as shown below:

```
/usr/local/bin/python3 /Users/Srikanth_Mohan/learning/learn_
tenacity.py
Retry forever with no wait if an IOError occurs, raise any
other errors
Retry forever with no wait if an IOError occurs, raise any
other errors
{'start_time': 330092.244061281, 'attempt_number': 2, 'idle_
for': 0, 'delay_since_first_attempt': 3.829202614724636e-05}
Process finished with exit code 0
```

We have covered some of the important features required for the retry mechanism using Tenacity. This library provides further functionalities like handling the retry failures using custom callbacks, adding logging before and after the retries, etc. It provides support retries for async services as well. I do hope this learning will come in handy in your work. END

By: Srikanth Mohan

The author is an Azure-certified cloud solution architect with 18 years of experience in cloud product validation and has a special interest in the reliability and scalability of cloud applications. He has worked on various digital transformation projects, IoT and connected devices initiatives, is currently with EPAM as Cloud Architect.